Chapter 07: Spring Security

Department of Software Engineering
Faculty of Information Technology (HMC-UTE)
Course: Web Programming
Course ID: WEPR330479
Lecturer: Msc. Khoa Hoang

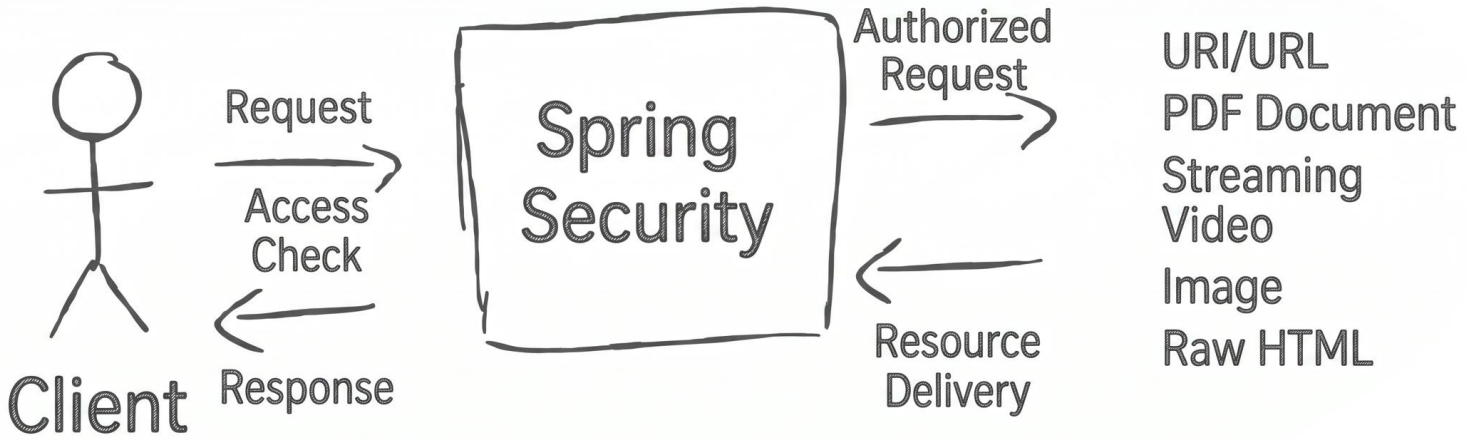
Content

- Web security fundamentals
- The security filter chain
- Security configuration

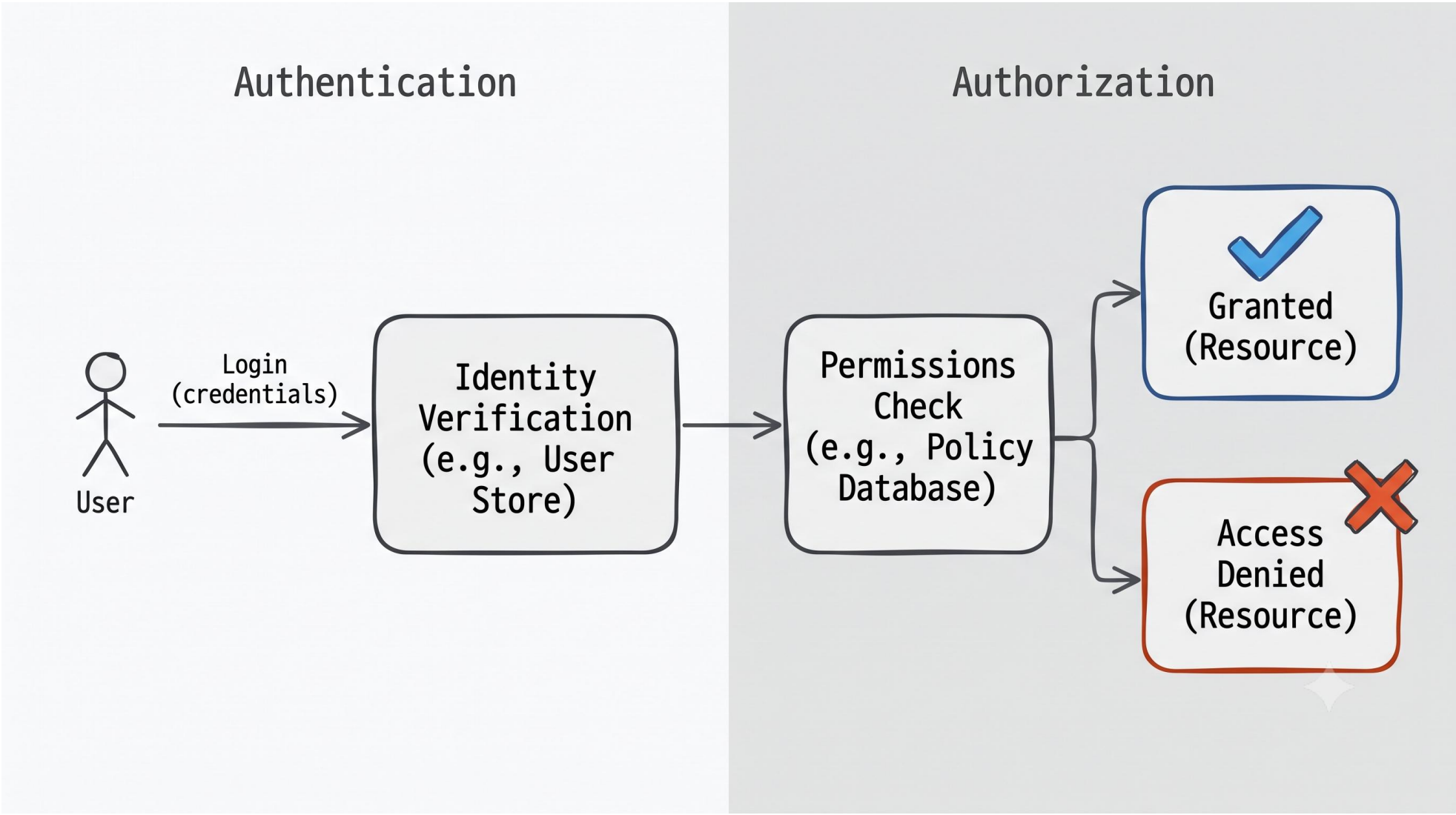
Web security fundamentals

Authentication, authorization and common threats

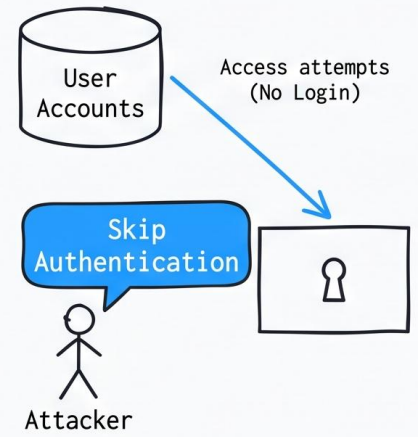
Overview


Client
Request
Access
Check
Response
Spring
Security
Authorized
Request
Resource
Delivery
URI/URL
PDF Document
Streaming
Video
Image
Raw HTML

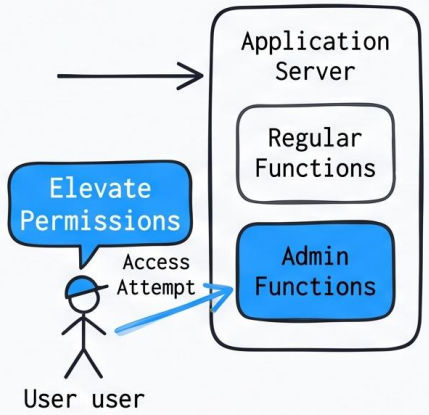
Authentication vs Authorization



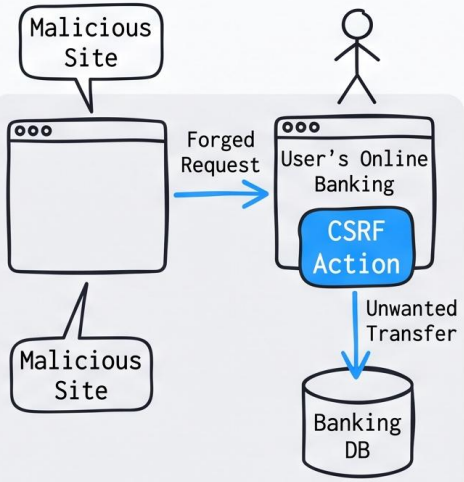
Common web security threats



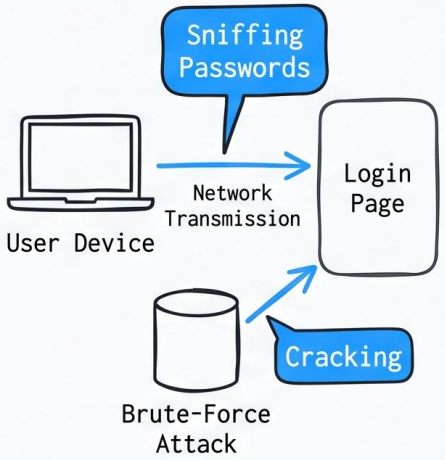
Unauthorized access: accessing restricted resources without login



Privilege escalation: regular user accessing admin functions



CSRF: malicious site tricking browser into forged requests



Credential theft: sniffing or cracking plain-text passwords

Note:
- Mitigate with parameterized queries
- Utilize CSRF tokens
- Help utilize unerenetrted resources/scority decames

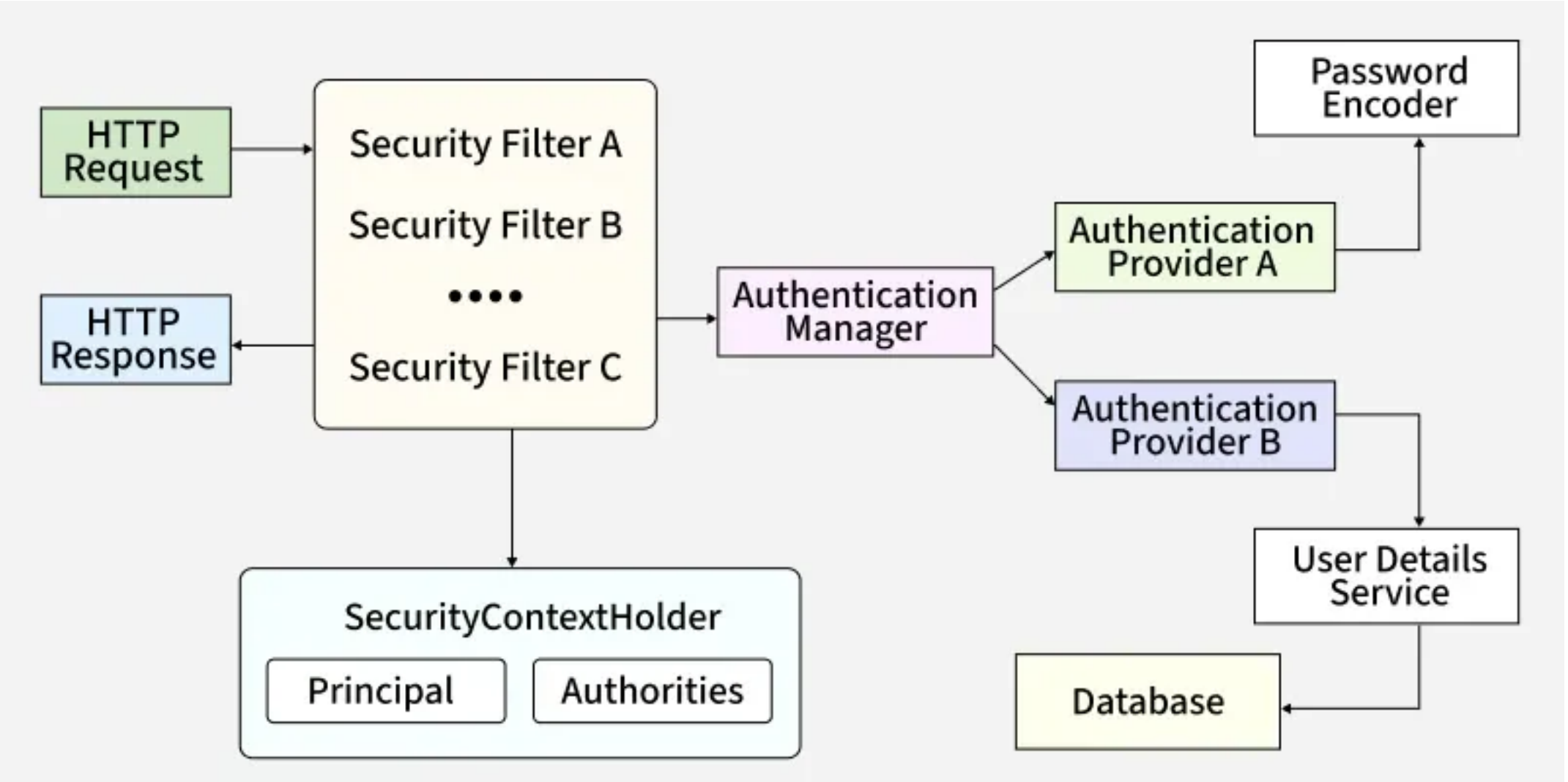
Spring security #1

- Spring Security acts as a powerful interceptor network built on standard Servlet Filters to protect your application via two main pillars: Authentication (verifying who you are) and Authorization (verifying what you are allowed to do)

- Before an incoming request ever reaches your Spring controllers, it must pass through a container-level filter chain.

- Official documentation: https://spring.io/projects/spring-security

Spring security #2



Spring security #3

- Spring MVC request flow
  Client -> DispatcherServlet -> Controller -> Service -> Repository -> Model -> DB

- When Spring Security is added, the flow becomes
  Client -> Spring Security Filter Chain -> Authentication -> Authorization -> DispatcherServlet -> Controller -> ...

- Requests are inspected before hitting Spring MVC controllers

Why run Security Filters before the DispatcherServlet?

- Resource protection: it is highly efficient. By blocking unauthorized requests at the very entrance, the application avoids spending CPU memory and parsing time inside DispatcherServlet or database lookups within controllers.

- Architecture decoupling: it separates infrastructural concerns (authentication, CORS, rate limiting) from your application's business logic

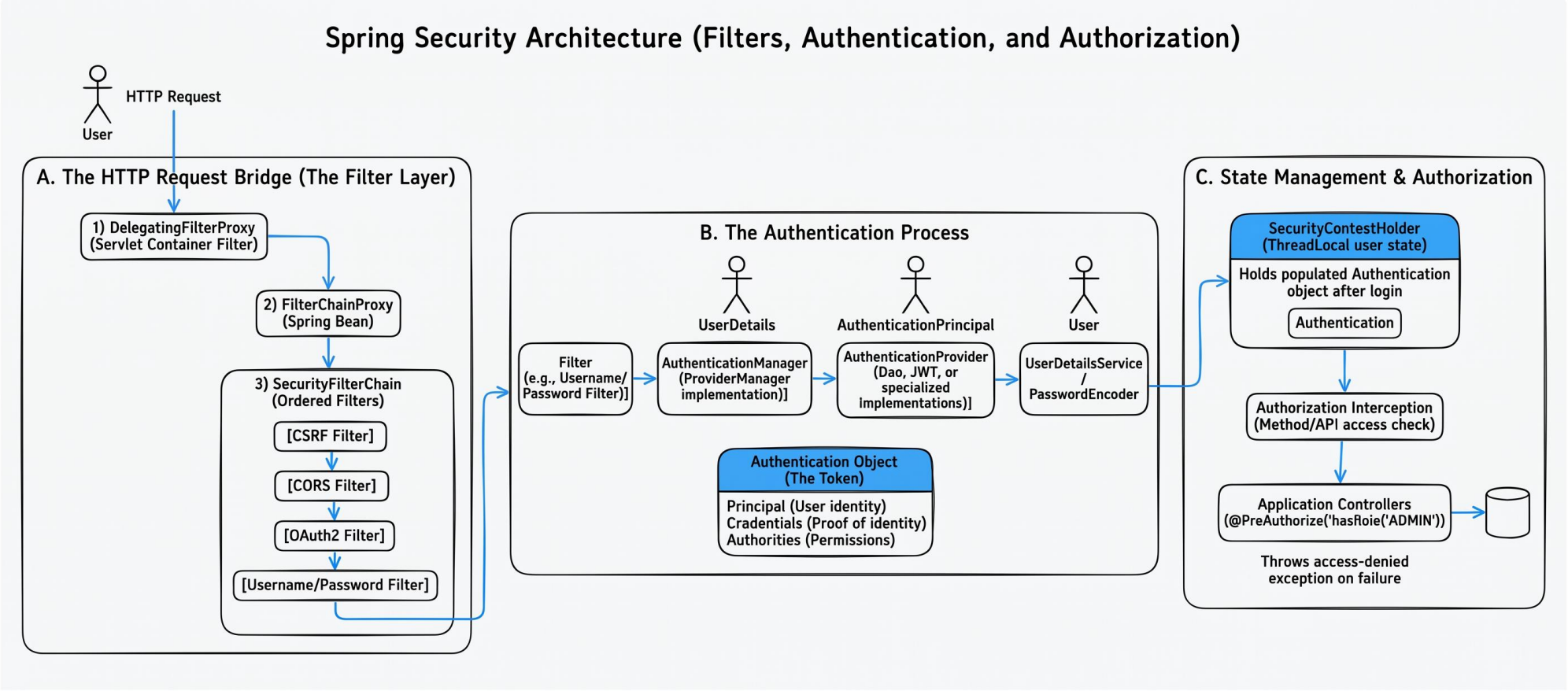
The security filter chain

Intercepting requests before reaching DispatcherServlet

Filter chain concept

- Spring Security is built on Servlet filters.
- Spring Security operates as a series of Servlet filters.
- **DelegatingFilterProxy** bridges Servlet container and Spring IoC
- **FilterChainProxy** manages the ordered list of security filters
- **SecurityFilterChain** consists of a list of specialized, ordered security filters. Each filter has one single, focused responsibility. The request passes through them sequentially
- If any filter rejects the request (e.g., a 401 Unauthorized or 403 Forbidden), the chain is broken immediately. The request is blocked, a response is returned to the client and the DispatcherServlet is never invoked.

Filter chain flow



- Spring Security is built on standard Servlet Filters.
- Authentication verifies identity; Authorization verifies allowed actions.
- Authorization is checked against GrantedAuthority permissions.
- Principal is the user, and Authorities are permissions.
- Help utilize unereneetried resources/security decames

Custom Authentication and Authorization

- Define custom UserDetailsService for database authentication.
- Create roles and authorities for users.
- Use @PreAuthorize and @Secured annotations for method-level security.
- Fine-tune URL-based access control.

Security configuration

Defining rules with SecurityFilterChain in Spring Boot 3+

Spring security configuration

- Configuring Spring Security in Spring Boot 3+ (Spring Security 6+) relies entirely on a component-based configuration using beans

- See more details on the Lab 4: Spring Security

Q&A – Thank you